Vector RAG vs Graph RAG

A Multi-Hop QA Benchmark on the ISO 20022 Payments Guide 2025

Final report - Freebuff Desktop project | August 2026

1. Project overview

This project compares two retrieval-augmented generation (RAG) architectures on a challenging 10-question multi-hop benchmark built from the 61-page ISO 20022 Payments Guide 2025 (Finland). Both systems answer with the same LLM and the same QA prompt - only the retrieval layer differs, so any difference in answer quality is attributable to the retrieval strategy.

2. What is used in this project

2.1 Document layer

- Source document: iso-20022-payments-guide-2025-en.pdf (61 pages), text-extracted to data/extracted_text.json (one entry per page).
- Questions: data/graph_rag_questions.json - 10 human-generated multi-hop questions (3 easy / 2-hop, 4 medium / 3-hop, 3 hard / 4-5-hop), each with a reference answer and an explicit reasoning chain. Every relationship in each chain is backed by the document.

2.2 Vector RAG

- Embeddings: sentence-transformers/all-mpnet-base-v2 (HuggingFace).
- Chunking: RecursiveCharacterTextSplitter, chunk size 1000 chars, overlap 200.
- Index & search: FAISS (langchain_community) cosine similarity, top-5 retrieval.
- FAISS index cached on disk (data/faiss_index) to skip rebuilds.

2.3 Graph RAG

- Graph stores: merged_knowledge.json (300 entities / 373 edges) and the sibling project's richer extraction - 602 nodes / 401 edges after normalization (aliases + descriptions + slug-id resolution).
- Graph library: NetworkX (MultiDiGraph) - in-memory traversal, no DB required.
- Retriever: keyword extraction -> entity matching (normalized-name + alias scoring) -> BFS traversal up to depth 3 collecting edges, multi-hop path chains and a reachable set -> entity-to-chunk mention index (every chunk that mentions each entity or alias) -> evidence-density chunk ranking -> context assembled as entity records, top chunks, path chains, then triples.
- Key tuning knobs (all global, no per-question rules): entity cap, keyword cap, chunk budget, per-page diversity (max 2 chunks/page), continuity fill for page-straddling passages, and a 15K-char context cap.

2.4 Evaluation

- LLM judge (same model as answering): per-question verdicts correct / partial / wrong, judged against the reference answer.
- Lexical metrics (metrics_v2): exact match, F1, answer accuracy, faithfulness, context precision / recall, hallucination rate, citation correctness, multi-hop success.
- Providers: Groq (llama-3.3-70b-versatile, llama-3.1-8b-instant), OpenRouter (nvidia nemotron-3-super-120b free / paid), OpenAI. Groq was the workhorse for the final runs (browser user-agent header required to pass Cloudflare).

3. The 10-question benchmark

All questions require 2-5 relationship hops across entities that are explicitly present in the guide (e.g. ChargeBearer/SLEV, UltimateCreditor, AOS2->ERI, VOP response types, pain.002.001.10 status codes,

the section-5.2 XML example). Reference answers and reasoning chains are included in the benchmark file.

3.1 Verdicts - sibling graph, llama-3.1-8b-instant

Q	Difficulty	Vector RAG	Graph RAG
1	easy	correct	correct
2	easy	correct	correct
3	easy	correct	wrong
4	medium	wrong	correct
5	medium	correct	correct
6	medium	partial	correct
7	medium	correct	correct
8	hard	wrong	partial
9	hard	partial	correct
10	hard	correct	wrong

Aggregate judge score (correct=1, partial=0.5, wrong=0): Vector RAG 0.70 vs Graph RAG 0.75 on this run (0.85 on the run before - Q3 is flaky on the 8B model). Across graph stores and models the stable picture is: graph RAG equals or beats vector RAG, and the gap widens with the denser sibling graph.

3.2 Results by graph store and model

Run	Vector	Graph
Merged graph (300 nodes) - Groq 70B	0.80	0.80
Merged graph (300 nodes) - nemotron 120B free	0.80	0.80
Merged graph (300 nodes) - Groq 8B	0.70	0.80
Sibling graph (602 nodes) - Groq 8B (prior run)	0.70	0.85
Sibling graph (602 nodes) - Groq 8B (this run)	0.70	0.75

Note: numbers from different models are not directly comparable; only same-model rows are apples-to-apples. The sibling (denser) graph consistently retrieves more relevant context: context_recall 0.73 vs 0.46 for vector on 8B.

4. Where Graph RAG clearly beats Vector RAG

Two questions where graph RAG answers fully and correctly while vector RAG fails - both are multi-hop / code-anchored questions where entity traversal matters more than lexical similarity.

4.1 Q4 (medium) - UltimateCreditor element

Question: "A payment is credited to the account of a financing company, but the ultimate creditor of the payment is a customer of that financing company. Which element in the Credit Transfer Transaction Information block (Block C) is used to identify this customer as the ultimate beneficiary?"

Reference: The UltimateCreditor element (index 2.148).

Vector RAG [wrong]

Named the Ultimate Creditor element but cited the wrong index (2.149) and wrong sub-elements - semantic search landed on a nearby element table row and the model copied the wrong identifiers.

Graph RAG [correct]

Named UltimateCreditor (index 2.148) with the correct definition - the entity record anchored to the exact element, and the traversal surfaced the authoritative definition sentence.

4.2 Q9 (hard) - payment status report schema / status

Question: "The payment message with MessageIdentification MSGID000001 passes structural validation at the bank. Which schema does the bank use for the return message, which Group Status does that return message carry, and which original identifier does it include?"

Reference: The Payment Status Report schema pain.002.001.10 with Group Status ACTC; it returns the original MessageIdentification MSGID000001 (in the OrgnlMsgId element).

Vector RAG [partial]

Got ACTC and MSGID000001 but invented a non-existent schema: "schema A: Structure, schema validation" - a hallucination from an adjacent validation section.

Graph RAG [correct]

Named the exact schema pain.002.001.10 (page 39), Group Status ACTC, and the original MSGID000001 - all three facts correct, with source-page citations.

Why the pattern: Q4 and Q9 ask for element/schema/status identifiers. Graph RAG walks entity links (UltimateCreditor, pain.002.001.10, ACTC, MSGID000001) to the authoritative page, while vector RAG retrieves whatever chunk is lexically closest - which can be an adjacent table row or a similar-looking section. The remaining shared failures (Q3 SHAR/SLEV conflation on 8B, Q8 five-page XML extraction) are model-synthesis limits, not retrieval gaps.

5. Known limits and honest caveats

- Q10 regression on the sibling graph: the richer context surfaces two valid status-report XML examples (page 50 group-level OrgnlGrpInfAndSts and page 51 transaction-level TxInfAndSts). The 8B model deterministically picks the wrong anchor - a synthesis failure; the correct evidence (AC01 + TxInfAndSts) is present and ranks higher (verified 3/3 trials).
- Q3 flakiness: the page-57 SLEV definition sentence is not in sibling-graph context (a pre-existing coverage gap), so 8B sometimes hedges; the merged graph covers it.
- Numbers are model-dependent: the 8B model compresses absolute scores. Direction (graph >= vector, denser graph >= merged graph) is consistent across Groq 70B, nemotron-120B and Groq 8B.
- No per-question rules were used anywhere; all retriever knobs are global (caps, weights, depth).

6. Artifacts

- benchmark_compare.py
- data/benchmark_results.json
- data/benchmark_results_sibling_8b.json
- data/sibling_vs_merged_8b.md
- data/benchmark_comparison.md
- data/graph_rag_questions.json
- data/merged_knowledge.json
- data/extracted_text.json
- data/faiss_index/